Deep Learning for Organoid Analysis: Graph-Based Modeling of 3D Cellular Architectures

Alexandre Martin

INRIA Sophia-Antipolis – Morpheme Team
In collaboration with IPMC Nice & Paris Cité University

ANR Morpheus Project

Thesis Defense

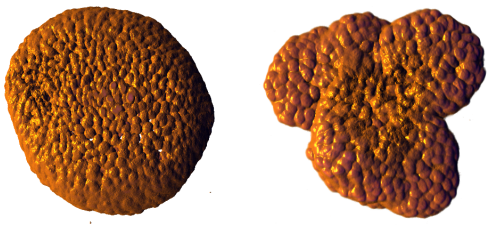
Organoids: Mini-Organs in a Dish

What are organoids?

- 3D structures grown *in vitro* from stem cells
- Self-organize to mimic real organ architecture
- Bridge gap between 2D cultures and animal models

Applications:

- Personalized medicine
- Drug screening
- Disease modeling



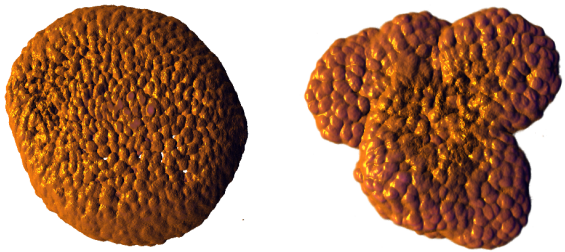
The Challenge: Automated Phenotype Classification

✔ Cystic (Healthy)

- Spherical, central cavity
- Organized structure
- **Uniform** cell distribution

⚠ Cauliflower (Perturbed)

- Irregular, multiple buds
- Disorganized structure
- **Clustered** cell distribution



Cell distribution differs between phenotypes

Goal

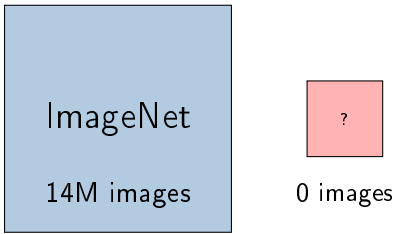
Automatically classify organoid phenotypes from 3D microscopy images

Starting Point: No Annotated Data

At the beginning of this thesis:

- ✗ No labeled organoid dataset
- ✗ No public benchmarks
- ✗ Expert annotation = expensive

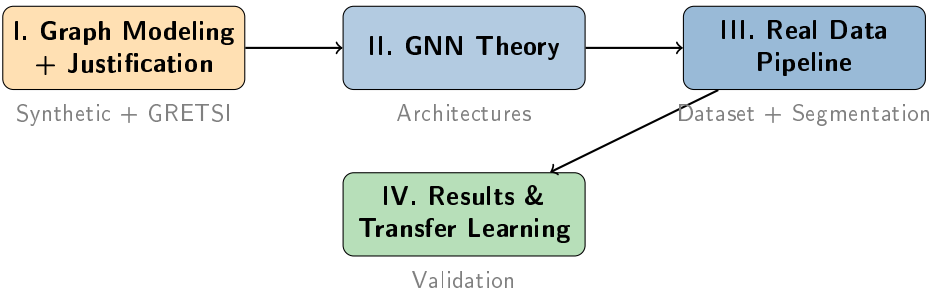
Key question:
How to develop and validate methods without real data?



Our Approach

Synthetic data + Graph modeling
to bootstrap the research

Presentation Outline



Narrative: No data → Synthetic + theory → Real data arrives → Transfer learning

Part I

Graph Modeling & Justification

Why graphs? Why GNNs?

Why Model Organoids as Graphs?

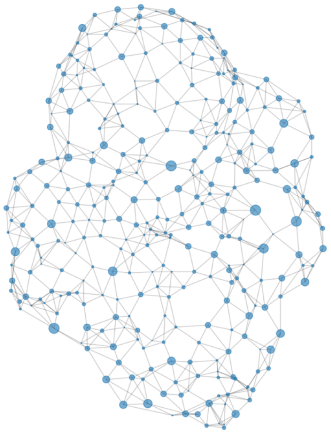
Key Biological Insight:
Cell **spatial organization** differs between phenotypes

Graph Abstraction:

- Each **cell** $\rightarrow$ **node**
- Spatial proximity $\rightarrow$ **edges**
- Node features: position (x, y, z), volume

Advantages:

- **1000× compression** (GB $\rightarrow$ MB)
- Captures relational structure
- Natural for point cloud data



Synthetic Organoid Generation: Motivation

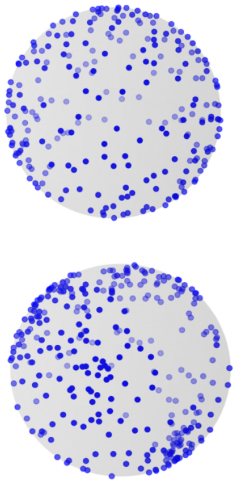
Problem: No real data at thesis start

Key Biological Insight:
Cell distribution differs between phenotypes

- Cystic → **uniform** distribution
- Cauliflower → **clustered** distribution

Solution: Spatial point processes

- Mathematical models for point patterns
- Controllable parameters
- Perfect ground truth labels



Uniform vs clustered distributions

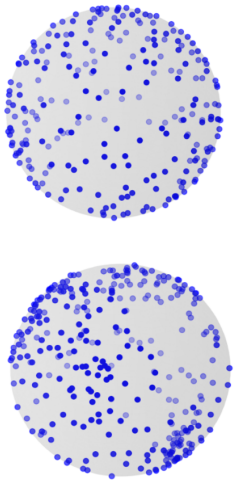
How We Simulate the Patterns

Cystic:

- Randomly scatter cells in the shape
- No clumps, just even spread

Cauliflower:

- Place cell groups (clusters) randomly
- Each group contains a few cells close together

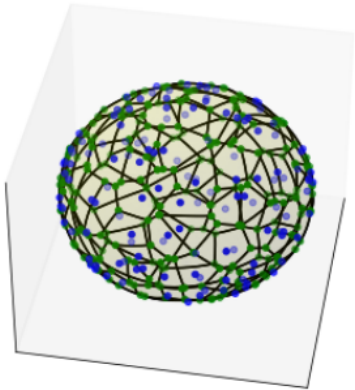


Uniform vs clustered distributions

Spherical Voronoi: Volumes & Cell Graphs

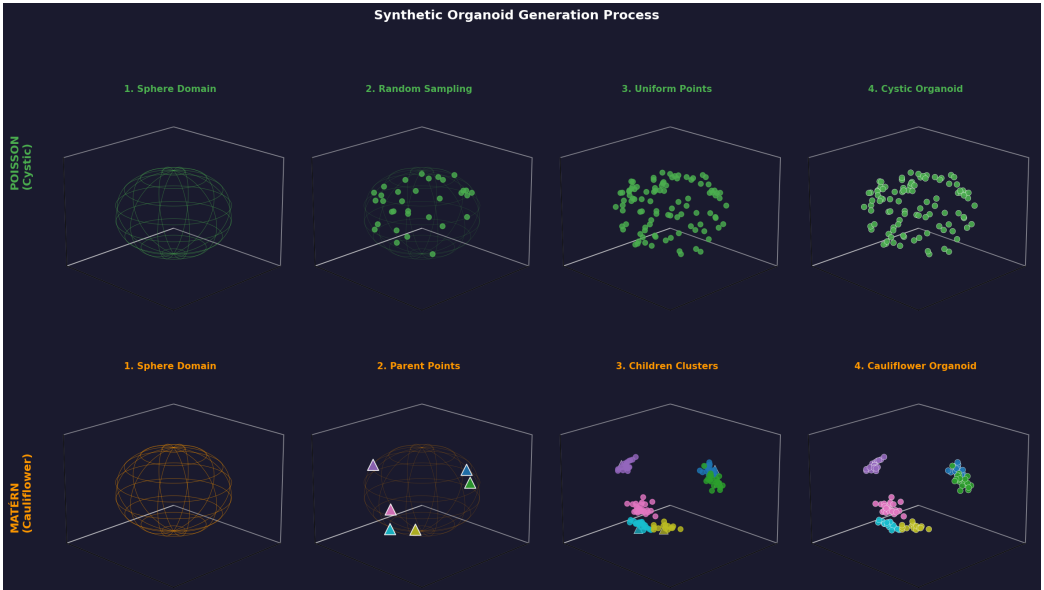
Synthetic Volume & Graph Generation:

- **Goal:** Assign realistic 3D cell volumes and build a cell interaction graph
- **Method:**
  - Scatter cell centers randomly on the sphere surface
  - Compute the **spherical Voronoi tessellation** (each region = one cell)
  - **Volume:** Use the volume of each region as the cell's feature
  - **Graph:** Connect an edge where Voronoi regions touch
- Automatically encodes realistic spatial constraints:
  - Volume is driven by packing geometry
  - Graph edges naturally reflect cell–cell contacts



Spherical Voronoi: Colors = cells, edges = touching regions

Synthetic Dataset



GRETSI 2025: GNNs vs Spatial Statistics

Research Question:

When should we prefer GNNs over classical methods?

Classical Approach:

- Ripley's K, F, G functions
- Random Forest classifier
- Well-established in spatial statistics

Deep Learning Approach:

- Graph Neural Networks
- Learn features from data
- End-to-end learning

Protocol:

- Synthetic data with controlled ground truth
- Test noise robustness and geometric generalization

What are Ripley's Functions?

Ripley's K, F, and G Functions

- **Purpose:** Quantify spatial patterns of point data—are points *clustered*, *random*, or *regularly spaced*?
- **How:** Measure how many points are within a distance r of each other, compared to randomness.

Key Functions:

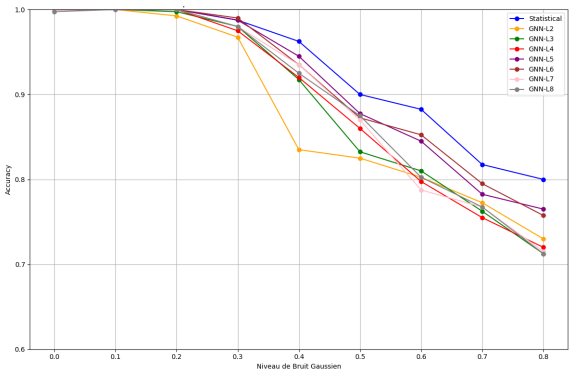
- $\mathbf{K}(r)$: Average number of other points within distance r of a typical point.
- $\mathbf{F}(r)$: Probability the nearest point from a random location is within r.
- $\mathbf{G}(r)$: Probability the nearest neighbor of a point is within r.

Interpretation:

Compare to random (Poisson) pattern:

- Above random $\rightarrow$ clustering
- Below random $\rightarrow$ regularity/repulsion

Result 1: Noise Robustness



Observation:
Spatial statistics are **more robust to noise**

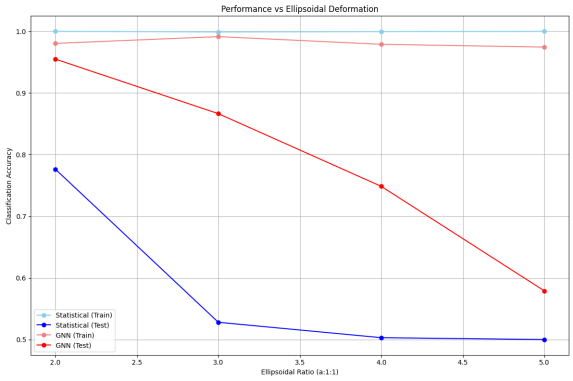
Why?

- Ripley's functions average over many point pairs
- Natural smoothing effect
- GNNs can propagate noise through layers

Optimal GNN depth: 5-6 layers

- Deeper $\rightarrow$ over-smoothing

Result 2: Geometric Generalization



Experiment:
Train on spheres, test on ellipsoids

Results:

- Spatial stats: 100% → 65% (−35%)
- GNNs: 95% → 82% (−13%)

Why?

- Ripley's K assumes isotropy
- GNNs learn **topological** patterns

Key Finding

GNNs excel when morphologies vary — **exactly our situation with real organoids!**

Justification: Why GNNs for Organoids

GNNs Excel When:

- Morphologies vary
- Shapes are irregular
- Geometry is non-spherical

$\Rightarrow$ **Real organoids**

Spatial Statistics Excel When:

- Geometry is perfectly spherical
- Consistent across samples
- Low noise environment

$\Rightarrow$ **Idealized conditions only**

Foundation for Our Approach

For cystic vs cauliflower with variable morphologies, **GNNs are the appropriate choice**

Part I Summary: Justification

1. **Graphs** naturally capture cellular spatial organization
2. **Synthetic data** enables method development without annotations
3. **GRETSI study** demonstrates:
   - GNNs $>$ spatial statistics for variable morphologies
   - Justifies our deep learning approach

Next

Now that we've justified GNNs, let's understand how they work

Part II

Graph Neural Networks

Theoretical Foundations

Deep Learning: The Big Picture

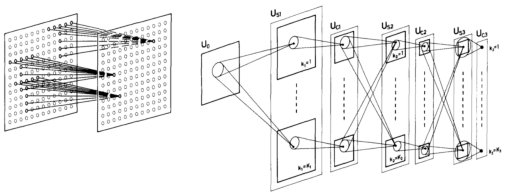
- **What is deep learning?**
  - Family of machine learning methods using **artificial neural networks** with many layers
  - Learns features from raw data, no need for handcrafted rules
- **Major successes:**
  - Computer vision (images), text understanding, speech, scientific data
- **Why is it powerful?**
  - **End-to-end learning:** from input to output, automatically
  - Learns complex nonlinear patterns
  - Improves with more data and scale

Where Did CNNs Come From?

Biological Inspiration:

- **Visual cortex** (Hubel & Wiesel, 1960s)
- Simple cells: detect **edges, orientations** in small receptive fields
- Complex cells: combine simple cells' outputs, sensitive to **patterns**

Key idea:
Hierarchical feature extraction from local to global



CNN: edge and feature detectors

CNNs Enable AI's Computer Vision Boom

- **2012:** AlexNet wins ImageNet challenge — breakthrough in image classification
- **Impact:** Superhuman performance in vision tasks (object detection, segmentation, etc.)
- **Applications:** From self-driving cars to medical imaging
- **Standard tool** for 2D natural image analysis

Why Not 3D CNNs?

Memory Constraints:

- Single organoid: $\sim$2 GB
- $2048 \times 2048 \times 200$ voxels
- GPU memory: prohibitive

Downsampling Issues:

- Destroys cellular details
- Individual cells unresolvable

No Native Invariance:

- Not invariant to 3D rotations
- Requires extensive augmentation

Grid-Based Limitation:

- Treats organoid as voxel grid
- Doesn't model cells as entities

Conclusion

Graphs + GNNs overcome all these limitations

From CNNs to GNNs: A New Paradigm

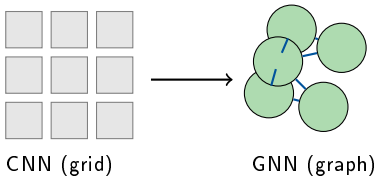
CNNs: Grids of pixels/voxels
⇒ Convolutions aggregate over local neighborhoods

BUT:

- Real organoids = collections of **cells**, not voxels
- **Spatial relations** matter (who's next to whom)
- Grids ignore true biological topology

GNNs: Process data as **graphs**

- **Nodes**: Cells
- **Edges**: Physical contact or spatial proximity
- **Features**: Position, volume, etc.



CNN: voxels *GNN: cells + relations*

GNNs: Message Passing Paradigm

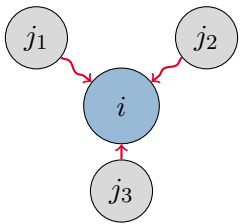
Core Idea: Nodes exchange information

Update Rule:

$$h_i^{(l+1)} = \text{UPDATE}\left(h_i^{(l)}, \text{AGG}\left(\{h_j^{(l)}\}_{j \in \mathcal{N}(i)}\right)\right)$$

Iterative process:

1. Each node sends messages to neighbors
2. Aggregate incoming messages
3. Update node representation



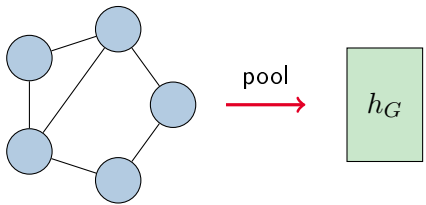
Messages flow to central node

From Node to Graph: Global Pooling

Goal: Graph-level prediction (classification), not node-level

Pooling Methods:

- **Mean:** $h_G = \frac{1}{|V|} \sum_{i \in V} h_i$
- **Max:** $h_G = \max_{i \in V} h_i$
- **Sum:** $h_G = \sum_{i \in V} h_i$
- **Attention:** learned weights



Our Choice: Mean + Max concatenation

Classification:

$$\text{Nodes} \xrightarrow{\text{GNN}} \text{Embeddings} \xrightarrow{\text{Pool}} h_G \xrightarrow{\text{MLP}} \hat{y}$$

GCN: Graph Convolutional Network (Baseline)

Kipf & Welling, 2017

Update Rule:

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{d_i d_j}} W^{(l)} h_j^{(l)}\right)$$

Properties:

- Normalized mean aggregation
- Simple and effective baseline
- ✗ Treats all neighbors equally
- ✗ No distance/importance weighting

GAT: Graph Attention Network

Veličković et al., 2018

Key Idea: Learn attention coefficients to weight neighbors

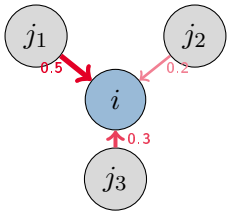
1. **Compute attention scores:**

$$\alpha_{ij} = \text{softmax}_j\left(\text{score}(h_i, h_j)\right)$$

2. **Weighted aggregation:**

$$h_i^{\text{new}} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \cdot W h_j\right)$$

✔ **Best performance** in our experiments



Learned attention weights

DeepSets: Set-Based Baseline

Zaheer et al., 2017

Approach: Treat organoid as unordered set of cells (ignore edges)

$$f(X) = \rho\left(\sum_{x \in X} \phi(x)\right)$$

Properties:

- Permutation invariant
- ✗ Ignores local spatial structure
- Each cell processed independently before aggregation

Observation: Performs reasonably well
⇒ Global statistics are informative, but **local structure helps**

Geometric Equivariance: Why It Matters

Problem:
Organoids have no preferred orientation in 3D culture

Invariance (for classification)

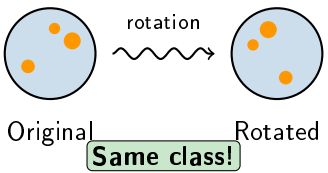
$f(R \cdot x) = f(x)$

Same prediction after rotation

Equivariance (for features)

$f(R \cdot x) = R \cdot f(x)$

Features rotate coherently



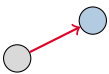
The E(3) Group: 3D Euclidean Symmetries

E(3) = Euclidean group in 3 dimensions

Rotations

Any 3D rotation around any axis

Translations



Shifting the entire structure in space

Reflections

Mirror symmetries

Key Insight

We can build architectures that **guarantee** these invariances by construction — not learned through augmentation, but **mathematically proven**.

EGNN: Equivariant Graph Neural Network

Satorras et al., 2021

Key Principle: Use only **invariant quantities** in messages

Message (uses distance = invariant):

$$m_{ij} = \phi_m\left(h_i, h_j, \|x_i - x_j\|^2\right)$$

Coordinate update (equivariant):

$$x_i^{new} = x_i + \sum_j (x_i - x_j) \cdot \phi_x(m_{ij})$$

Advantages:

- ✔ No augmentation needed
- ✔ Guaranteed robustness
- ✔ Better with less data

Trade-off:
Slightly lower raw accuracy

Architecture Comparison Summary

Architecture	Aggregation	E(3)-Equiv.	Strength
GCN	Mean	✗	Simple baseline
GAT	Attention	✗	**Best accuracy**
DeepSets	Global sum	✗	No local structure
EGNN	Distance-based	✓	**Guaranteed robustness**

Key Trade-off:

- GAT $\rightarrow$ Best raw performance
- EGNN $\rightarrow$ Guaranteed geometric invariance

Part II Summary: GNN Theory

1. **Message passing** enables learning on graphs
2. **GAT** with attention achieves best accuracy
3. **EGNN** provides guaranteed E(3)-equivariance
4. **All architectures** outperform spatial statistics on variable morphologies

Next

Real data finally arrives — how to build the processing pipeline?

Part III

From Real Images to Graphs

Processing Pipeline

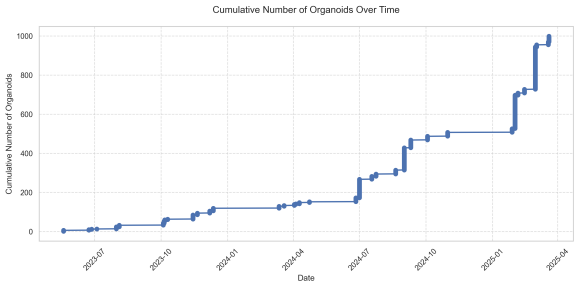
The Real Dataset Arrives

ANR Morpheus Project

- 22 months of data collection
- May 2023 – February 2025
- IPMC Nice + Paris Cité

Dataset Statistics:

- 1,311 imaged samples
- 2,272 extracted organoids
- **500 well-differentiated** selected for training



Data accumulated over 22 months

Challenge

How to transform 2 GB 3D images into graphs for GNN processing?

Standardized Acquisition Protocol

Culture Conditions:

- Analysis at day 7 (J7)
- Nuclear + specific markers
- 3D suspension culture

Imaging Parameters:

- Confocal microscopy
- 20× or 40× magnification
- 8-bit TIFF format
- Resolution: 2048×2048×(100-300)

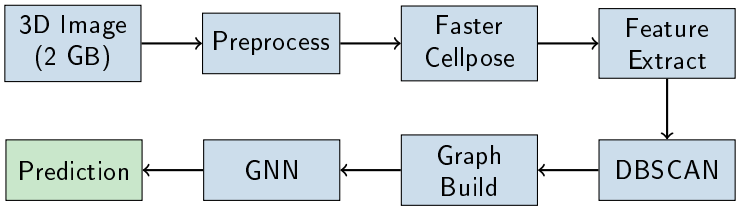
Cell Statistics:

- Min: ∼20 cells/organoid
- Max: ∼5,000 cells/organoid
- **Mean: ∼250 cells**

Storage:

- Raw images: ∼1 TB
- Compressed graphs: ∼200 MB
- **5000× compression!**

End-to-End Processing Pipeline



Processing time: ∼20 min/organoid
(dominated by segmentation)

Compression: GB → MB
(**1000× reduction**)

The Segmentation Bottleneck

Cellpose = State of the Art

- Best accuracy: F1 = 0.98
- Robust to size/shape variations
- Pre-trained models available

But Prohibitively Slow:

- 30 sec/slice (GPU)
- 300 slices/organoid $\approx$ **2.5 hours**
- 1,000 organoids $\Rightarrow$ **2,500 hours**
- Full dataset (2,272 org.) $\Rightarrow$ **237 days!**

Need for Optimization

Even with 10 GPUs: **3+ weeks** of compute!

Our Two Contributions:

1. Geometric ellipse fitting
2. Faster Cellpose

Contribution: Geometric Ellipse Fitting

Idea: Model nuclei as ellipses

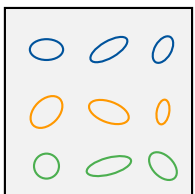
(No deep learning, inspired by marked point processes)

2D Detection Algorithm:

1. Gaussian blur + Black-Hat morphology
2. Detect local maxima (candidate centers)
3. Test bank of elliptic filters:
   Angles θ, axes a, aspect ratios r
4. Select ellipse maximizing response:

$$R = \frac{(K * I)(x, y)}{|\epsilon|}$$

5. Adaptive threshold (70% of max)
6. Handle overlaps by shrinking



Bank of elliptic filters

Filter Bank Parameters:

Angles:	$\theta_i = i\pi/I$
Small axis:	3–25 px
Aspect ratio:	1.0 to 3.0

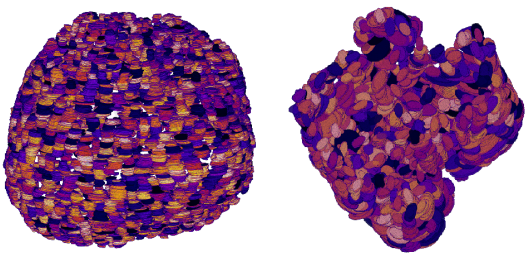
Ellipse Method: 3D Reconstruction

3D Matching Algorithm:

1. For each ellipse ϵ_n in slice z
2. Find candidates in slice $z + 1$
3. Compute combined distance:
4. Match if distance small enough
5. Merge into 3D objects

Key Properties:

- ✔ Deterministic (no training)
- ✔ No GPU required
- ✔ 10× faster than Cellpose



3D reconstruction from stacked ellipses
Each color = one matched cell

Cellpose: How It Works (State of the Art)

Principle:

- Predicts a **gradient field**
- Each pixel "points" toward its cell center
- **Flow tracking** groups pixels into instances

Why Instance Segmentation?
Graphs require **discrete nodes**
$\Rightarrow$ Each cell = unique label

Performance:

- F1 = 0.98 (best accuracy)
- But: 30 sec/slice
- $\Rightarrow$ Need optimization!

Architecture:

- U-Net encoder-decoder
- ResNet backbone
- Two output branches: X, Y, Z gradients

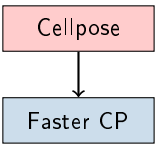
Contribution: Faster Cellpose

Goal: Match Cellpose accuracy, much faster

How: Our "Faster Cellpose" combines three optimizations:

- **Smaller model:** Reduced layers and channels to cut compute and memory, while retaining key U-Net structure.
- **Weight pruning:** Remove redundant weights (prune to 50% sparsity) to speed up inference and shrink the model size.
- **Mixed-precision:** Use float16 instead of float32 for faster computation on modern GPUs, with little accuracy loss.

Result: 5× faster, small F1 drop (0.98 → 0.95)

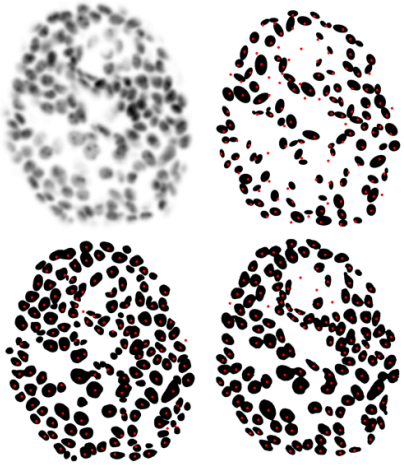


Speed-up: **5×**
F1: 0.95

Segmentation: Key Performance Trade-offs

Method	F1	sec/slice	GPU
Cellpose	0.98	30	Yes
Faster CP	**0.95**	**6**	Yes
Ellipses (ours)	0.88	3	No
StarDist	0.85	5	Yes

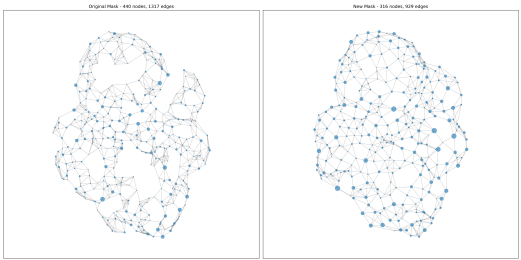
Choice: Faster CP
Best speed/accuracy, GPU-friendly



Impact of Segmentation on Graph Construction

Downstream Impact:
Segmentation quality directly affects the reconstructed cellular graph.

- **Accurate segmentation** → correct cell count, positions, and connectivity.
- **Over/under-segmentation** leads to spurious or missing nodes and edges.
- Structural graph features (*e.g.*, degree, clustering) can be severely distorted.
- Error propagation: graph-based phenotyping/modeling relies on initial accuracy.



Example: segmentation quality modulates resulting organoid graph

DBSCAN: Separating Multiple Organoids

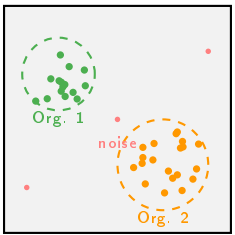
Problem:
Each image may contain **multiple organoids**
($\sim$1.7 organoids/image on average)

Solution: DBSCAN Clustering

- Density-based spatial clustering
- ✔ No need to specify # clusters
- ✔ Handles irregular shapes
- ✔ Identifies noise/debris

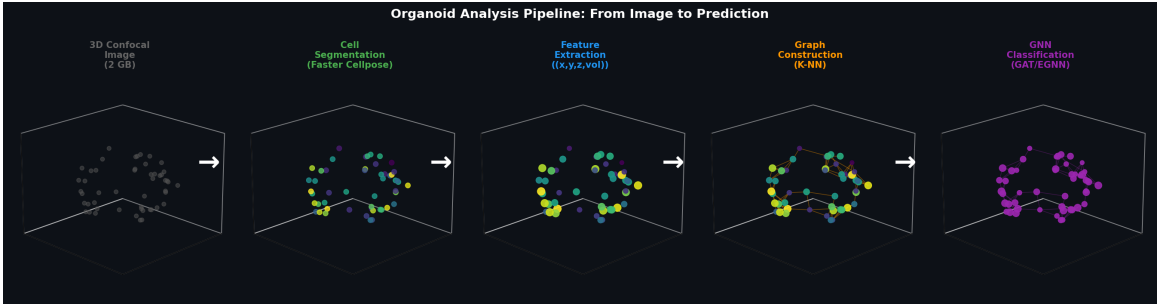
Filtering Criteria:

- Min: 20 cells (exclude debris)
- Max: 5,000 cells (exclude aberrant aggregates)
- Reject border-touching organoids



DBSCAN separates organoids

Graph Construction from Segmented Cells



Pipeline Performance: Timing Breakdown

Per-Organoid Processing:

Step	Time
Preprocessing	< 1 sec
Faster Cellpose	**$\sim$20 min**
Feature extraction	< 1 sec
Graph construction	< 1 sec
GNN inference	< 0.1 sec
Total	**$\sim$20 min**

Segmentation dominates (>99%)

Scalability:

- Each organoid: independent unit
- Trivial parallelization
- Batch GNN inference on GPU

1000 Organoids:

- Sequential (1 GPU): 330h ($\sim$14 days)
- Parallel (20 GPUs): 17h

Hardware:
CPU: Intel Core i7
GPU: NVIDIA RTX 3080

Part III Summary: Real Data Pipeline

1. **Real dataset:** 500 well-differentiated organoids (22 months)
   - Cystic vs Cauliflower phenotypes
   - Standardized acquisition protocol
2. **Preprocessing:** Percentile normalization + denoising
3. **Faster Cellpose:** 5× speedup (F1 = 0.95)
   - Knowledge distillation + pruning + FP16
4. **DBSCAN:** Separates multiple organoids per image
5. **Hybrid K-NN graphs:** 1000× compression, 4D features

Next

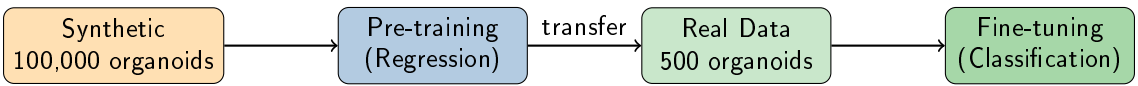
Transfer learning: can synthetic pre-training help with real data?

Part IV

Results & Transfer Learning

From Synthetic to Real

Transfer Learning: Synthetic → Real



Phase 1: Pre-training

- 70,000 synthetic organoids
- Task: Regress Matérn parent number
- Learn spatial organization patterns

Phase 2: Fine-tuning

- 500 real organoids
- Reinitialize classification head
- Reduced learning rate (10^{-4})

Performance on Synthetic Data

Task: Regression of Matérn parent number
Test set: 15,000 synthetic organoids

Architecture	MSE	vs GCN
GCN (baseline)	0.198	—
DeepSets	0.145	+27%
EGNN	0.137	+31%
GAT	**0.118**	**+40%**

Key findings:

- GAT wins: attention learns which neighbors matter
- EGNN: great results with equivariance

Synthetic Results: Analysis

Why GAT wins:

- Attention learns **which neighbors matter**
- Adaptive to clustering patterns
- Multi-head captures different relationships

DeepSets' good performance:

- Global statistics are informative
- But local structure still helps
- GAT > DeepSets confirms this

EGNN trade-off:

- Slightly behind GAT in raw MSE
- But: guaranteed equivariance

Performance on Real Data

Dataset: 500 well-differentiated organoids
Evaluation: 5-fold cross-validation

84% Accuracy

(GAT + pre-training)

Cauliflower:

- Precision: 93%
- Recall: 74%

Main confusion: Low-deformation cauliflowers

Cystic:

- Precision: 78%
- Recall: 95%

Transfer Learning Impact

Strategy	Accuracy
GAT from scratch	76%
GAT pre-trained	**84%**
Improvement	**+8%**

Data Efficiency:

- Pre-trained + 25% data: 78%
- From-scratch + 100%: 76%

$\Rightarrow$ **4× fewer annotations needed!**

Learning Curves: Detailed Analysis

Data Efficiency Gains:

Data %	From-scratch	Pre-trained
10%	58%	71% (+13%)
25%	67%	78% (+11%)
50%	72%	82% (+10%)
100%	76%	84% (+8%)

Key Observation:
Largest gains at **low data** regime

Convergence Speed:

- Pre-trained: **20-30 epochs**
- From-scratch: 80-100 epochs
- $\Rightarrow$ **3× faster**

Implication:
Synthetic pre-training teaches useful representations of spatial organization

Practical Impact:
125 real organoids (pre-trained) $\approx$ 500 real organoids (from-scratch)

Computational Efficiency

Inference Performance:

- **200+ organoids/minute**
- GPU batch processing
- Enables high-throughput screening

Memory Footprint:

- ∼8 GB GPU memory
- Compatible with standard hardware
- RTX 3080 / A100

Reproducibility:

- **100% deterministic**
- No inter-observer variability
- Identical results every run

Scalability:

Organoids	Time
100	∼30 sec
1,000	∼5 min
10,000	∼50 min

vs Manual Analysis

Manual: 30 min/organoid × 1000 = 500 hours
Automated: 5 min total — **6000× faster**

Identified Limitations

- **Segmentation dependency:** Errors propagate through pipeline
- **Validation scope:** Prostate organoids only; generalization needs testing
- **No inter-laboratory validation:** Different microscopes, protocols
- **No interpretability analysis:** Which cells drive predictions?

Part IV Summary: Results

1. **GAT achieves best performance:**
   - 84% accuracy on real data
   - MSE 0.118 on synthetic data
2. **Transfer learning is transformative:**
   - +8% accuracy improvement
   - 4× data efficiency
   - 3× faster convergence
3. **Practical system:**
   - 200+ organoids/minute inference
   - Fully automated, reproducible

Conclusion

Contributions & Perspectives

Contribution 1: Graph-Based Representation

Geometric Graphs for Organoid Modeling

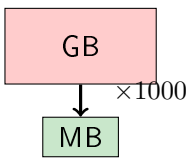
From raw 3D images to compact graph representations

Key Innovation:

- Cell $\rightarrow$ Node abstraction
- Spatial proximity $\rightarrow$ Edges
- 4D features (position + volume)

Impact:

- **1000× compression**
- Preserves biological structure
- Enables GNN analysis



Contribution 2: GRETSI Comparative Study

Published at GRETSI 2025

Rigorous comparison: GNNs vs Spatial Statistics

Key Findings:

- GNNs: better **geometric generalization**
- Spatial stats: better **noise robustness**

Recommendation:

GNNs for variable morphologies
Statistics for idealized conditions

Impact:

- Guides method selection
- Justifies deep learning approach
- Foundation for thesis methodology

Citation:

Martin et al., GRETSI 2025

Contribution 3: Synthetic Data Generation

Point Process-Based Generation

Poisson + Matérn processes for synthetic organoids

Method:

- Poisson $\rightarrow$ Cystic (uniform)
- Matérn $\rightarrow$ Cauliflower (clustered)

Scale:

- **100,000** synthetic organoids
- Perfect ground truth labels
- Controllable difficulty

Impact:

- Enabled research without real data
- Foundation for transfer learning
- +8% accuracy improvement

Generalizable:
Applicable to other spherical/ellipsoidal biological structures

Contribution 4: Transfer Learning Strategy

Synthetic → Real Transfer

Pre-train on synthetic, fine-tune on real

Results:

+8%	4×	3×
Accuracy improvement	Data efficiency	Faster convergence

Practical Impact

125 organoids (pre-trained) = 500 organoids (from-scratch)

Transforms annotation requirements

Contribution 5: Complete Automated Pipeline

End-to-End System

From raw 3D images to phenotype predictions

Components:

- Preprocessing
- Faster Cellpose (5× speedup)
- Graph construction
- GNN classification

Properties:

- **Fully automated**
- **200+ organoids/minute**
- **Open-source**
- Reproducible

Short-Term Perspectives

Methodological Extensions:

- Multi-scale graphs
  (cell → region → organoid)
- Graph Transformers
  (global attention mechanism)
- Interpretability analysis
  (which cells drive predictions?)

Multi-Modal Integration:

- Spatial transcriptomics + imaging
- Temporal data from time-lapse
- Metabolomic features

Clinical Validation:

- Prospective patient cohorts
- Multi-site testing

Long-Term Vision

Therapeutic Response Prediction:

- Patient-derived organoids
- Test treatments in silico
- Personalized therapy selection

Generative Graph Models:

- Use GNN-based generative models (e.g. GraphVAEs, diffusion models) to simulate realistic 3D organoid architectures

Multi-Organ Extension:

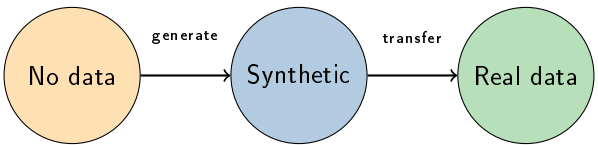
- Brain organoids
- Liver, kidney, lung
- Tumor organoids

Societal Impact:

- **3Rs principles**
  Replace, Reduce, Refine
- Accelerate drug discovery
- Reduce animal testing

Final Message

Geometric Graph Neural Networks
are a powerful tool for 3D organoid analysis



Thank you for your attention.
I am happy to take your questions.